

Skill VM: Write Once, Run Everywhere Efficiently

Le Chen, Erhu Feng, Yubin Xia✉, Haibo Chen
Institute of Parallel and Distributed Systems, Shanghai Jiao Tong University

Abstract

LLM agents increasingly adopt skills as a reusable unit of composition. While skills are shared across diverse agent platforms, current systems treat them as raw context, causing the same skill to behave inconsistently for different agents. This fragility undermines skill portability and execution efficiency.

To address this challenge, we analyze 118,000 skills and draw inspiration from traditional compiler design. We treat skills as code and LLMs as heterogeneous processors. To make portability actionable, we decompose a skill's requirements into a set of primitive capabilities, and measure how well each model–harness pair supports them. Based on these capability profiles, we propose SkillVM, a compilation and runtime system designed for portable and efficient skill execution. At compile time, SkillVM performs capability-based compilation, environment binding, and concurrency extraction. At runtime, SkillVM applies JIT code solidification and adaptive recompilation for performance optimization.

We evaluate SkillVM across ten LLMs of varying scales and four agent harnesses, covering SkillsBench and representative skill tasks. Results demonstrate that SkillVM significantly improves task completion rates across different models and environments while reducing token consumption by up to 40%. In terms of performance, SkillVM achieves up to 3.2× speedup with enhanced parallelism, and 19–50× latency reduction through code solidification.

CCS Concepts: • **Computing methodologies** → **Intelligent agents**; • **Software and its engineering** → **Runtime environments**.

Keywords: LLM Agents, Agent Skills, Skill Optimization

ACM Reference Format:
Le Chen, Erhu Feng, Yubin Xia, Haibo Chen. 2026. Skill VM: Write Once, Run Everywhere Efficiently. In *ACM SIGOPS 32nd Symposium on Operating Systems Principles (SOSP '26), September 29-October 02, 2026, Prague, Czech Republic.* ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/3830418.3843874



This work is licensed under a Creative Commons Attribution 4.0 International License.
SOSP '26, Prague, Czech Republic
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2585-2/2026/09
https://doi.org/10.1145/3830418.3843874

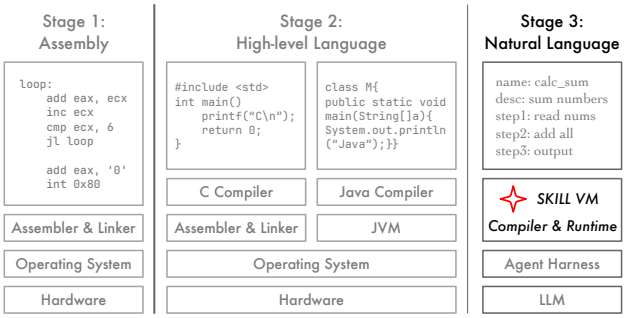


Figure 1. Evolution of programming abstractions. Skills are the frontier: natural language programs of hundreds of lines that lack a compiler and runtime for cross-target portability.

1 Introduction

LLM-based agents [1, 12, 50–52, 71] are reshaping how software tasks get done. Instead of writing code by hand, a developer describes a goal, and the agent reasons, invokes tools, and delivers the result [57, 77]. To support this paradigm, a growing ecosystem of *skills* [6, 7, 27, 67] has emerged to make agent execution more reliable and practical. Skills primarily consist of natural language descriptions and scripts, encapsulating domain-specific procedures and best practices [8]. Over 100,000 skills are now distributed across major platforms [19, 67], spanning data analysis, finance, office automation, programming, and beyond.

However, current agents' support for skills is simplistic: skills are treated as additional context and passed directly to the model. Models differ substantially in their ability to understand and execute skills [31, 41]. Across eight models spanning a range of scales, enabling skills degrades performance on 15% of tasks overall (e.g., 7% for Opus 4.6 and 25% for Qwen3-30B). For another 17% of tasks, scores remain unchanged (excluding tasks with 100% completion), and on up to 87% of tasks, at least one model shows no improvement after skill usage. Recent work [31] has reached similar conclusions, finding that on the SWE-Benchmark, 39 out of 49 skills showed no score improvement, and 3 skills experienced significant score degradation.

We attribute these issues to two factors: (1) even when a skill is loaded, the model may ignore its guidance during execution; and (2) even when the model follows the skill, the skill's assumptions about model capabilities may not match the model's actual competence. Beyond differences in task completion gains, the semi-structured code snippets and workflow-style formats common in current skills also incur substantial token overhead (e.g., 451% token increase

while pass rates remain unchanged [31]) and higher latency during execution.

This problem arises from a fundamental mismatch between static skills and the variability of underlying models and agent harnesses. The capabilities a skill demands may not align with the capabilities of the LLM invoked at runtime. Revisiting the evolution of computing paradigms [21, 68, 70] (as shown in Figure 1), we observe that in the agent era, **skills are code**, and **LLMs are processors**. Yet today, no mechanism exists to efficiently and reliably execute skills across heterogeneous LLMs and agent harnesses.

Inspired by how classical computing systems handle code, we introduce SkillVM, a compilation and runtime system for skills that enables efficient cross-model execution and provides unified underlying support for skill execution. Through SkillVM, skills are compiled into forms tailored to different models, allowing each model to better understand and execute skills. Furthermore, SkillVM provides a unified runtime environment that systematically manages skill loading, parsing, and concurrent execution.

We structure SkillVM around classical compilation techniques: interpreted execution [36], ahead-of-time (AOT) compilation [4, 63, 64], and just-in-time (JIT) optimization [13, 14, 21, 24, 35, 69]. Currently, agents handle skills using only interpreted execution, feeding raw skill text directly to the model. This approach hampers skill portability and execution efficiency. In contrast, SkillVM further applies AOT and JIT compilation to optimize for diverse backend models and execution environments:

AOT compilation: Skills inherently encode requirements of model capabilities, environment configuration, and parallelism opportunities. Therefore, SkillVM analyzes these requirements before execution and optimizes skills accordingly. *Capability-based compilation* characterizes the capability gap between LLMs and skill requirements by extracting 26 primitive capabilities. Each capability abstracts a distinct dimension of model behavior with multiple proficiency levels. The compiler measures the target's model and harness against these capability dimensions and adapts the skill specification to better align with the target's strengths and limitations. Second, *environment binding* extracts implicit dependencies on packages and tools from skill descriptions and generates setup scripts executable at load time, ensuring all dependencies are satisfied before skill execution. Finally, *concurrency extraction* identifies parallelism opportunities in skills at three granularities: script-level, tool-level, and agent-level parallelism. It explicitly exposes these opportunities to the agent harness, thereby improving task execution efficiency.

JIT optimization: At runtime, SkillVM leverages runtime information to continuously optimize skill performance and correctness. *Code solidification:* Skills may contain parameterized script templates which may be instantiated multiple times during different executions. JIT compilation materializes these high-frequency templates into instantiated executable code, bypassing LLM parsing. *Adaptive recompilation:* JIT monitoring tracks model execution and adaptively recompiles skills when capability gaps emerge, ensuring skills can self-improve through iterative optimization.

After skill compilation completes, the skill runtime parses compiled skill artifacts (including optimized skills, instantiated scripts, and concurrency dependency graphs) rather than simply passing raw text to the model. Additionally, the skill runtime coordinates with the system's available resources and tool capabilities to schedule agent execution in real time, and ensures stable and reliable agent execution.

We evaluate SkillVM across ten LLMs of varying scales and four agent harnesses, covering SkillsBench [41] and representative skill tasks (118 tasks). Results demonstrate that SkillVM significantly improves task completion rates (averaging 15.3%) across different models and agent harnesses [12, 52]. Furthermore, for tasks that can be completed, SkillVM reduces token consumption up to 40%. In terms of performance, SkillVM achieves 3.2×–50× wall-clock speedups through fine-grained parallelization and JIT code solidification. Additionally, we have completed AOT compilation of the top 1,000 popular skills across eight models, including Qwen3 [73] and DeepSeek-V3.2 [43].

2 Skills in the Wild

In this section, we first introduce skills, the agent that uses them, and how skills are invoked in practice. We study the skill ecosystem at scale through more than 110,000 skills collected from two major distribution platforms: `clawhub.ai` [19] (28,990 skills) and `skills.sh` [67] (89,280 skills). Based on this ecosystem analysis, we further conduct experiments to identify concrete problems in current skill usage.

2.1 Skill Definition

A *skill* is a distributable, self-contained knowledge pack that enhances an AI agent's capability on a specific class of tasks [2, 6]. A skill may introduce knowledge the base model does not have, or constrain the model toward a prescribed workflow when it already has the relevant capability but would otherwise improvise. For example, a PDF-processing skill [7] teaches the model how to use `pdfplumber` for table extraction, and also constrains the model to use `pypdf` instead of the deprecated `PyPDF2` when merging PDF files.

A skill is typically defined in a structured file comprising three layers: (1) *metadata*—a name, description, and trigger conditions that allow the agent runtime to discover and select the skill automatically; (2) *instructions*—the main body of a skill that encodes multi-step workflows, tool references, and

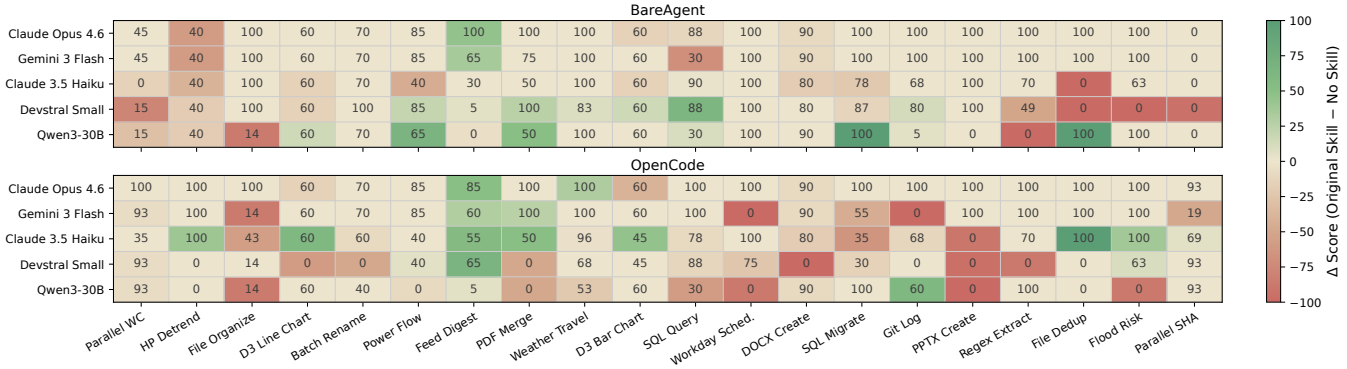


Figure 2. Skill performance across models and harnesses. Columns are tasks, rows are models, and each subplot corresponds to one harness. Cell colors indicate score deltas relative to the no-skill baseline, and cell labels show absolute task scores. Both model identity and harness choice significantly affect outcomes.

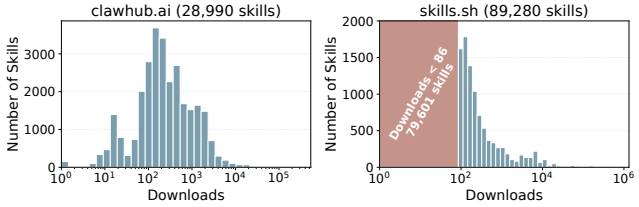


Figure 3. Skill download distribution on `clawhub.ai` and `skills.sh`. Both platforms show a long-tailed distribution.

other task-specific guidance in natural language; and (3) *bundled resources*—scripts, references, and templates stored in companion files.

This structure distinguishes skills from ad-hoc prompt snippets. The metadata makes a skill a discoverable, selectable unit, enabling the agent runtime to match tasks to applicable skills automatically rather than requiring manual prompt assembly [9]. Compared with fine-tuning, skills require no weight modification and can be operated entirely at the prompt level [16, 44], making them lightweight to distribute and easy to compose. Multiple agent platforms have converged on this abstraction [2, 9, 20, 22, 29, 50], and existing tools support basic skill authoring, management, and optimization.

2.2 LLM Agent and Agent Harness

An LLM agent works in a ReAct loop [77]. It receives a task, reasons about what to do, issues a tool call (reading a file, running a command, making an API request) [49, 62], observes the result, and reasons again. This loop repeats until the task is done.

The agent does not interact with the outside world directly. It runs inside an *agent harness*, a runtime framework that sits between the model and the operating environment [40, 72]. The harness manages the LLM API connection, registers the tools the agent can call, dispatches those calls, and controls how much conversation history fits in the context window. Claude Code [12], OpenCode [52], and OpenClaw [51] are examples of widely used harnesses.

Harnesses differ in ways that matter for skills [75]. Some provide a rich set of file-system and web search tools, while others expose only basic read, write, and shell execution operations. Some can spawn independent sub-agents that work concurrently, while others only support sequential execution. These differences directly affect whether a skill's instructions can be carried out.

2.3 Characterizing Skills

Ecosystem Scale and Distribution. The skill ecosystem is large but uneven. Figure 3 shows the download counts across both platforms. On `skills.sh`, 89% of the 89,280 skills have fewer than 86 downloads, while a handful of head skills reach 10^4–10^6. `clawhub.ai` follows a similar long-tailed pattern, peaking around 10^2–10^3 downloads. Over 118,000 skills exist across the two platforms, but the vast majority see little use. The long tail consists primarily of skills published by various organizations and services, for example how to use their APIs and tools, serving smaller but legitimate audiences.

Skill Taxonomy. Skills differ substantially in what they provide the agent with. We use LLM-assisted analysis to classify 15,063 skills that have more than 100 downloads into three categories, based on which aspect the skill most emphasizes: tool reference (52%), procedural guidance (28%), and content generation (20%).

Tool-reference skills teach the model how to operate specific tools, APIs, or CLIs. They are essentially usage documentation loaded into the agent's context. A representative example is a skill that documents how to use python to merge, split, and manipulate PDF files.

Procedural skills prescribe step-by-step workflows and reasoning strategies for carrying out a task. A representative example is a debugging skill that enforces a four-phase process: reproduce the failure, trace the root cause, apply a targeted fix, and verify the fix with tests.

Generative skills ask the model to produce content, code, documents, or prose. The core quality depends on the model's own capability. An example is a frontend-design skill [7] that

generates production-grade web components with specific design guidelines and styling conventions.

Tool-reference and procedural skills together account for 80% of the ecosystem. These skills encode analyzable structure, including script fragments and step-by-step workflows. The correctness of using these skills depends on whether the agent follows steps, not on model's generative ability.

Workflow Structure. Beyond task type, skills also exhibit repeated internal structure. Across the 15,063 classified skills, 76% contain explicit procedural structure: numbered steps, conditional branches, and data dependencies between steps.

Code Fragments. 75% of skills embed code-like fragments: shell commands, API call patterns, or script snippets whose overall shape recurs across invocations while only the input-specific parameters change. For example, a weather skill [51] may provide a family of curl command templates for current conditions, forecasts, and format options, where the command structure stays fixed while parameters such as the city, output format, and forecast day vary.

2.4 Problems with Current Skill Usage

Skills are loaded into the agent's context as raw text, with no adaptation. But skills carry implicit assumptions about the model, agent harness, and user environment. When those assumptions are wrong, three failure modes emerge.

P1: Model Mismatch. A skill implicitly assumes that the model is capable enough to follow its instructions. In practice, models differ dramatically [45]. Figure 2 shows scores for a subset of tasks across five models and three harnesses.[1] Performance varies substantially across models on the same skill. Moreover, enabling skills degrades performance on 15% of tasks; leaves scores unchanged on 17% of tasks (excluding tasks with a 100% completion rate); and yields no improvement for at least one model on 87% of tasks.

Case Study: the pptx-creation skill [7] recommends `PptxGenJS` (a javascript library) for slide generation. Claude-opus-4.6 and gemini-3-flash both score 100 with this skill, but devstral-small misreads `PptxGenJS` as a CLI when using OpenCode and repeatedly executes wrong commands. Without the skill, devstral-small instead uses `python-pptx` and scores 95. The skill assumes a model can distinguish a library API from a CLI, which holds for frontier models but not weaker ones.

P2: Harness Mismatch. The same model produces different results on the same task depending on the agent harness it runs in [75]. Figure 2 also shows this dimension: across the heatmap, harness-induced variance is comparable to model-induced variance. Skills are written without knowledge of what tools and plugins the harness provides.

[1] BareAgent is a minimal agent harness that we built. It implements only basic agent runtime logic and exposes only read, write, and exec tools.

Case Study: Gemini 3 Flash scores 100 on the workday-scheduling task with the original skill on BareAgent, but 0 on OpenCode with the same skill. The failure is triggered by a malformed JSON key that makes the output unparseable. BareAgent adds only a minimal system prompt, whereas OpenCode prepends extensive tool documentation, and the longer combined context induces the formatting error.

P3: Environment Mismatch. Skills have dependencies on the execution environment, but the user's machine may lack the required packages, tools, or configurations. Across the 15,063 classified skills, 53% require at least one external package or tool. Among them, 19% declare no dependencies at all, and a further 40% list only a subset of the packages and tools that their instructions and scripts actually use. Even when prerequisites are listed, the instructions assume a generic machine rather than the user's actual setup. The real requirement depends on the current OS, hardware, installed versions, and package-manager state. Figure 4 shows the effect on two representative tasks. When the required library is absent, both Qwen models drop to 33–67% success rate while generating 2–4× more output tokens on failed workarounds. Even Claude Opus 4.6, which still succeeds on every run, generates 56–69% more output tokens because it must diagnose the mismatch and install the missing package at runtime. Leaving diagnosis to the model therefore turns every missing dependency into a repeated tax on both correctness and efficiency.

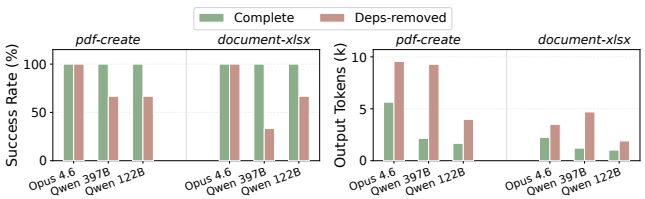


Figure 4. Removing a required dependency hurts both correctness and efficiency.

2.5 Challenges

Skills are distributed as plain Markdown so that any agent can load them with no special tooling. The mismatches above undermine this portability promise. Restoring portability requires adapting each skill to its target, which is challenging for two reasons.

C1: Targets Differ Along Many Axes. Each of the three dimensions above, model, harness, and environment, varies independently. A skill that needs adaptation along one dimension may need a different adaptation when two dimensions change together. The fix for a weak model on a rich harness differs from the fix for the same model on a minimal one. This interaction makes the adaptation space combinatorial: it cannot be covered by a fixed set of rewrite rules.

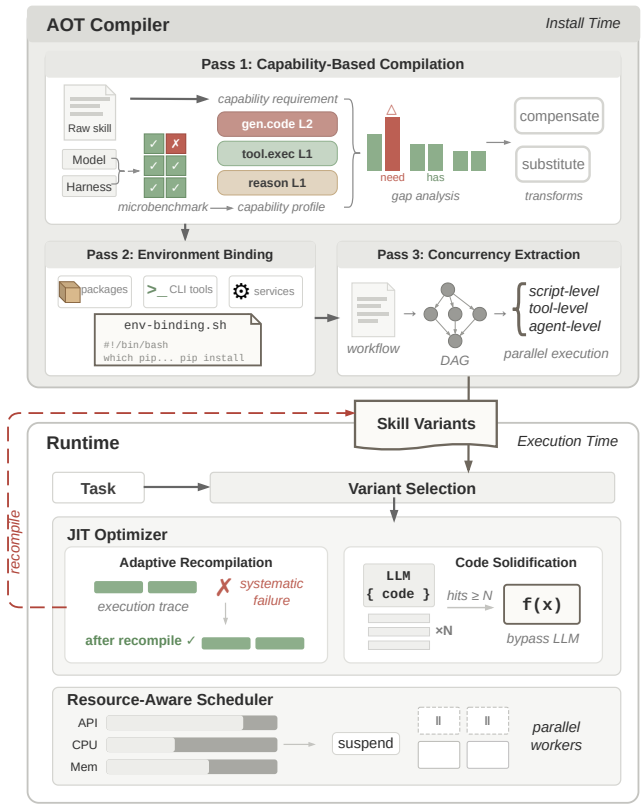


Figure 5. SkillVM architecture. The AOT compiler produces optimized skill variants at install time through three passes. The runtime manages variant selection, JIT optimization, and resource-aware scheduling during execution.

C2: Skills Are Unstructured Natural Language. Source code requires compilation before it can run on a target system, and has a grammar, types, and a well-defined AST that tools can analyze [4]. In comparison, skills are written in natural language. Although skills are mostly created following common conventions, different skills vary widely in structure, terminology, and level of detail. Before any adaptation can happen, the skill's implicit requirements and workflow structure must be recovered from prose.

3 Design Overview

SkillVM is a compilation and runtime system for skills. It enables skills to be portable across heterogeneous targets by combining compile-time specialization with runtime adaptation. Figure 5 shows the architecture.

Like classic programming language systems, SkillVM transforms a source representation, a skill, against a target, generates target-specific artifacts, and optimizes skill execution using runtime feedback.

However, the source representation of a skill fundamentally differs from conventional programs. Traditional language systems operate on programs with formally defined syntax and semantics, whereas skills are expressed in natural language. Consequently, syntax-directed compilation alone is insufficient. SkillVM therefore uses an LLM as its semantic transformation engine and organizes optimization into two stages: AOT compilation and JIT optimization.

AOT Compiler. At skill installation time, the compiler analyzes the skill against the target and produces one or more optimized *skill variants*. The compiler performs three distinct passes: *Capability-based compilation* (§4.1) handles model and harness mismatch (P1–P2) by adapting the skill to the capabilities the target's model and harness provide. *Environment binding* (§4.2) handles environment mismatch (P3) by materializing the dependencies required for execution. *Concurrency extraction* (§4.3) identifies parallelism latent in the skill's workflow and maps it to the target's harness.

Runtime & JIT optimizer. When a task arrives, the runtime selects the variant compiled for the current model–harness pair and loads it through the standard progressive disclosure mechanism. Two components then operate on top of normal execution. The *JIT optimizer* (§5.2) monitors execution outcomes across invocations and triggers recompilation when it detects capability gaps that the AOT compiler missed. It also compiles structurally fixed code patterns into executable functions that bypass the LLM inference. The *resource-aware scheduler* (§5.3) bridges compile-time parallelism annotations and run-time resource availability, throttling or suspending concurrent sub-agents when demand exceeds capacity.

4 Skill Compilation

When a user installs a skill for the first time, SkillVM compiles it against the target (model, harness, host environment) before the skill ever runs. The compiler reads the raw skill text, identifies the target, and produces one or more optimized *skill variants* through three sequential passes. Each pass closes a different kind of gap between what the skill expects and what the target provides.

4.1 Capability-Based Compilation

The first pass addresses the model and harness mismatch, two of the three target dimensions. The environment dimension is deferred to the second pass (§4.2). Because targets vary widely (§2.5), the compiler cannot embed optimization logic specific to each target. Instead, SkillVM introduces *primitive capabilities*, an abstract vocabulary for expressing what a skill requires and what a target can provide. The compiler profiles each target offline with microbenchmarks for these capabilities, compares the profiling result with the skill's capability requirements, and selects the optimization strategy based on the gap.

4.1.1 Primitive Capabilities

Primitive capabilities define the basic abilities a skill requires from its target. A primitive capability is an indivisible unit that describes what is needed to complete the tasks a skill defines. It is defined purely from the demand side, independent

Table 1. Representative primitive capabilities and their proficiency levels. The full catalog contains 26 primitive capabilities across four categories.

Primitive	L1	L2	L3
`gen.code.shell`	Basic commands (ls, cat)	Pipes, redirection, loops	Complex pipelines (sed, awk)
`reason.arithmetic`	Single-step ops	Multi-step	Compound
`tool.exec`	Single command	Params and relative paths	Chained multi-step execution
`follow.procedure`	3 sequential steps	5–7 with branches	Loops and verification

of any specific model. The definition of primitive capabilities follows three principles:

- *Composability*: any concrete task within a skill can be described as a combination of primitive capabilities.
- *Generality*: each primitive capability must be broad enough to appear across many skills, keeping the total capability set small and the compiler's workload bounded.
- *Semantic independence*: a primitive capability concerns structural correctness, rather than whether the output is insightful or well-argued.

We derived the primitive capability set from a corpus of 15,063 skills in two stages. First, we curated 50 representative skills spanning document generation, script execution, data analysis, and code development, and used LLM-assisted analysis guided by the three principles above to decompose them into an initial set of 19 primitive capabilities. We then manually reviewed each primitive capability to verify that it satisfies all three principles. Second, we validated the initial set against the full corpus of 15,063 skills by checking whether each skill's requirements can be expressed as a combination of the existing primitives. Skills that could not be covered revealed missing capabilities. A missing capability was added to the set only when it appeared in at least 1% of the corpus, filtering out rare or domain-specific requirements. This process converged on 26 primitive capabilities that cover the requirements of 95% of the skills.

We also observe that different tasks demand the same primitive capability at different depths. We therefore define multiple proficiency levels for each primitive capability. A higher level represents deeper use of the capability: an *L(n+1)* requirement cannot be satisfied by *L(n)* ability. Table 1 shows representative examples. For each primitive capability level, we generate a suite of microbenchmarks and evaluate them across different models. If a model succeeds on higher-level microbenchmarks but fails lower-level ones, we treat the hierarchy as miscalibrated, revise the level definitions, and repeat the evaluation.

The capability set and level boundaries are extensible design choices rather than fixed assumptions of the compiler. As skills evolve, new primitives or levels can be incorporated without changing the compiler architecture.

4.1.2 Measuring the Gap Between Skill and Target

Before applying optimizations, the compiler must determine the gap between what the skill requires and what the target

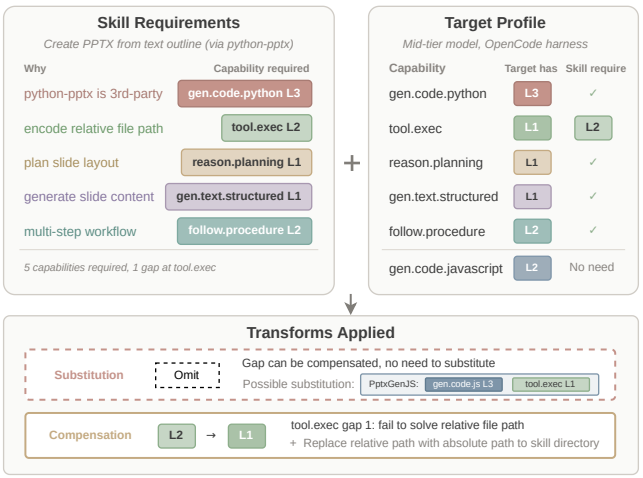


Figure 6. Capability-based compilation on a PPTX skill. The compiler extracts skill requirements (left), profiles the target (middle), and applies transforms based on the gap (bottom).

provides. On the skill side, the compiler extracts the skill's capability requirements at install time: it decomposes the skill into concrete tasks and maps each task to the primitive capabilities it requires.

On the target side, SkillVM profiles the target with microbenchmarks generated for each primitive capability to measure how well the target supports it. Success on a microbenchmark for a primitive capability at a given level indicates that the model has attained at least that level of proficiency for that capability. This profiling is performed when the user sets up the agent and model. The results are cached and reused for all skill compilations for the same target.

4.1.3 Capability-Aware Skill Transforms

With the skill's capability requirements and target's capability profile in hand, the compiler walks through each capability requirement and selects a transform based on the gap between the required level and the target's profiled level.

Compensation applies when the target supports the required capability but at a lower level. The compiler identifies the distinctions between the two levels of a primitive capability. It then transforms the skill accordingly to reduce its capability requirement from a higher level to one supported by the target. Such transformations may include providing examples, making instructions more explicit, and strengthening task constraints. Compensation is the preferred transform because it preserves the skill's original intent.

Substitution is the fallback when compensation is not viable: the target lacks the capability entirely, or the level gap is too large for compensation to bridge reliably. The compiler switches to an alternative implementation path that achieves the same goal using different capabilities. For example, if a skill requires `gen.code.python` at L3 for a `pandas` workflow but the target only reaches L1, the compiler can switch to an SQL-based path if the target's `gen.code.sql` reaches L2.

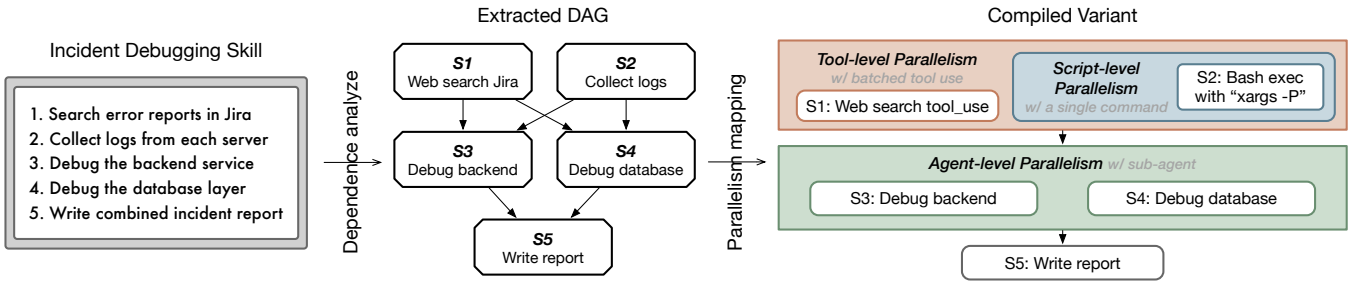


Figure 7. Concurrency extraction on an incident-response skill. The compiler decomposes sequential steps into a workflow DAG, identifies parallel opportunities both between steps and within steps, and maps each to a concurrency primitive.

These alternative paths form *equivalence classes* that the compiler identifies during requirement extraction.

Case Study. Figure 6 illustrates the compilation process on a real skill that teaches an agent to create PPTX presentations using `python-pptx`. The skill requires five primitive capabilities at varying levels. The target matches four of them directly but provides `tool.exec` at only L1, while the skill requires L2 for resolving relative file paths.

The compiler identifies a possible substitution solution: switching to `PptxGenJS` via `gen.code.javascript` would replace `python-pptx` and lower the `tool.exec` requirement to L1. However, because the `tool.exec` gap is only one level, compensation is preferred. The L2 profiling results reveal that this model fails to resolve relative file paths when executing commands. The compiler then replaces relative paths with absolute paths rooted at the skill directory.

4.2 Environment Binding

The second pass resolves *P3: environment mismatch*. Rather than merely checking whether dependencies are installed, the compiler uses a built-in *env-binding skill* to reconcile the skill's assumptions against the host environment and generate an env-binding script.

At skill installation time, the compiler first extracts a dependency manifest from the output skill of the first pass and any explicit prerequisites, covering external libraries, CLI tools, and system services. If the skill author provides an explicit dependency description, the compiler uses it as additional evidence rather than treating it as complete: 59% of skills that use an external dependency declare their dependencies incompletely or not at all (§2.4), and a static list alone cannot determine the appropriate installation actions for a specific host. For each manifest entry, the compiler runs a lightweight presence check (e.g., pip show, which). Dependencies that are already satisfied require no further work. When the check fails, the compiler performs deeper, system-aware probing to determine how to resolve the dependency. The compiler then emits an idempotent env-binding script which will check and repair the dependency before skill execution. When the environment changes or the env-binding script fails, execution falls back to the model, and the script's output is passed to the model as additional context.

4.3 Concurrency Extraction

The third pass finds parallelism hiding in plain sight. Recall that 76% of skills contain procedural structure (§2.3). These skills describe multi-step workflows in sequential prose, but not every step actually depends on the one before it.

The compiler uses LLM-assisted analysis to decompose the skill into discrete steps, identifying for each step what inputs it consumes and what outputs it produces. It then builds a workflow DAG: nodes are steps, and an edge from step A to step B exists when B consumes an output of A. The compiler validates the DAG before emitting it: it must be acyclic and free of dangling nodes. A DAG that fails validation is discarded and the affected steps remain sequential. The compiler also analyzes each step internally for sub-operations that can proceed concurrently. Figure 7 illustrates this process on an incident-response skill. For each parallel opportunity, whether between steps or within a step, the compiler selects a concurrency primitive based on the task structure and the harness. SkillVM defines three levels of parallelism, named after where the concurrency materializes: in scripts, in tool calls, or in sub-agents.

Script-Level Parallelism. A single step applies the same operation to multiple independent data items, such as running the same analysis on each of 15 CSV files. The compiler rewrites the step to process items concurrently using language-level parallelism primitives appropriate to the skill's implementation, such as shell-level parallel execution, Python's multiprocessing, or JavaScript's Promise.all. Script-level parallelism requires no special harness support.

Tool-Level Parallelism. Multiple independent steps each need a tool call with no data dependency between them, such as running several independent code-analysis scripts on a project. The compiler groups the DAG nodes into one parallel stage and rewrites the skill so that tool invocations are issued in a single LLM turn, instead of one after another in sequential prose. The compiler annotates the skill with which step outputs should be bound back to which downstream DAG inputs after the batch completes. Tool-level parallelism requires the harness to support batch tool dispatch.

Agent-Level Parallelism. The workflow decomposes into independent sub-tasks that each require multi-turn reasoning,

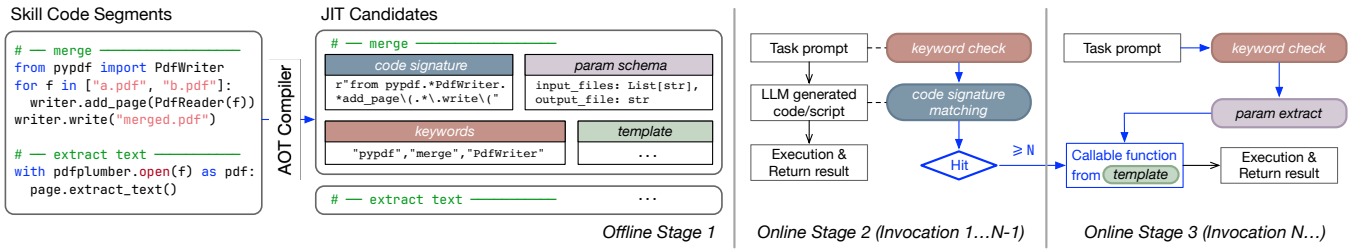


Figure 8. Code solidification pipeline. The AOT compiler analyzes skill code segments and generates JIT candidates with code signatures and templates (Stage 1). The runtime checks that the model's generated code matches the predicted signature across consecutive invocations (Stage 2). After promotion, a compiled function replaces LLM inference (Stage 3).

such as debugging three independent services. The compiler rewrites the skill to extract each sub-task into a separate sub-agent block with an explicit task description, declared input context, and expected output that can be consumed by the parent workflow. The annotation marks this block for sub-agent dispatch, so the runtime spawns one sub-agent per block in its own session and merges the returned outputs back into the DAG. Agent-level parallelism requires harness support for sub-agent spawning.

Opportunities whose required parallelism support is unavailable on the target's harness fall back to sequential execution. For each viable opportunity, the compiler either rewrites the skill's instructions to execute items concurrently at the script level or emits annotations in skills that make the parallel structure explicit.

5 Skill Runtime with JIT Optimization

AOT compilation generates multiple skill variants for different targets. Consequently, at runtime, SkillVM loads the skill variant compiled for the current target. Since AOT compilation is performed only once per skill and target, the compilation overhead is modest.

However, some problems only surface at execution time, such as capability gaps that static profiling missed or resource contention among parallel sub-agents competing for rate-limited APIs. Other opportunities, like code patterns that recur across invocations, only become visible after repeated runs. The SkillVM runtime addresses both through JIT optimization (§5.2) and a resource-aware scheduler (§5.3).

5.1 Skill Registration and Execution

Compilation produces multiple skill variants for each skill, along with the env-binding scripts generated by environment binding (§4.2). When a new task arrives, the runtime selects the variant compiled for the current model–harness pair. From the agent's perspective, the complexity of multiple variants is invisible: it sees a single set of skills and loads them through the same progressive disclosure mechanism described in §2. Before loading the full skill context, the runtime will first execute the env-binding script to verify whether runtime dependencies are satisfied.

5.2 JIT Optimization

Beyond AOT compilation, which tailors primitive capabilities to each target, SkillVM further applies JIT optimization to enhance execution quality and efficiency through two complementary mechanisms: adaptive recompilation and code solidification.

5.2.1 Adaptive Recompilation

The runtime tracks the outcome of every task executed with a skill. When a skill execution fails (e.g., failed tool call or task incompletion) or retries during the agent loop, the runtime records a structured failure or retry log. Although the model may perform self-correction through reflection, this correction is not incorporated into the skill, allowing the same error to recur in subsequent executions.

When the same skill exhibits recurrent failures across invocations, the runtime determines whether each failure is task-specific or systematic. A failure is task-specific if it arises from the input or constraints of one task, and systematic if the same cause can recur across tasks. Only systematic failures trigger adaptive recompilation: the runtime feeds the accumulated failure logs and the model's self-recovery traces to the compiler, which applies targeted compensation transforms to the skill. A revised variant ships only if it improves the measured score without regressing other tasks, and the original skill is kept when no round qualifies.

5.2.2 Code Solidification

Recall that 75% of skills contain solidifiable code (§2.3): code whose overall structure is fixed, varying only in input-specific parameters. Normally, the LLM re-executes the full reasoning and tool-use cycle on every invocation, spending tokens and latency on a process whose structure is identical each time. Code solidification identifies these repetitive patterns and promotes them into executable code that bypasses model inference. Figure 8 illustrates the three-stage pipeline.

In the first stage, the AOT compiler analyzes code and script segments within each skill, identifying whether they contain parameterized patterns eligible for solidification. For eligible segments, it generates a *JIT candidate* containing four components: keywords that filter relevant invocations, a *code signature* that captures the expected generated-code

structure as a match pattern, a code template with parameter slots, and a parameter schema describing the extractable arguments. These candidates are bundled with the compiled skill variant.

In the second stage, the runtime monitor watches LLM invocations across repeated calls. For each invocation, it first checks keywords for relevance, then matches the generated code against the code signature. The code solidification is triggered only after the code signature matches successfully across multiple consecutive invocations, ensuring that the code structure generated by the LLM remains consistent with the patterns analyzed in the AOT stage. If the model's output consistently diverges from the code signature, the system stays on the LLM path and never promotes.

In the third stage, the template is instantiated into a callable function: the template code is wrapped with parameter extraction and output handling to produce a standalone shell script or code function. This instantiation step is lightweight and mechanical, as the AOT stage has already done the analysis. Subsequent invocations bypass the LLM entirely: the runtime extracts parameters from the task context, calls the instantiated function, and returns the result. If solidified code fails to execute or raises a runtime exception, SkillVM demotes it and re-enables LLM-based code generation.

5.3 Resource-Aware Parallel Scheduling

During concurrency extraction (§4.3), the AOT compiler identifies where parallelism is possible, but how much parallelism is *efficient* depends on conditions that change at run time: CPU usage, external API rate limit, the machine's available memory, and other shared resources.

SkillVM dynamically monitors system state and schedules task execution accordingly, bridging the gap between compile-time opportunity and run-time reality. For API-bound workloads, it watches response latencies and HTTP 429 (rate limit) signals. For compute-bound workloads, it monitors CPU and memory usage. When pressure rises above a configured threshold, SkillVM applies two mechanisms to mitigate resource contention. First, it throttles the launch of new sub-agents to avoid introducing additional concurrency pressure. Second, for sub-agents that are already running, SkillVM selectively suspends a subset of them (i.e., pauses their agent-loop execution), based on per-agent resource usage or launch order, to reduce competition for shared resources.

6 Implementation

We implement SkillVM in 36,200 lines of TypeScript. SkillVM consists of three major components: a profiler, an ahead-of-time (AOT) compiler, and a just-in-time (JIT) optimizer.

The profiler provides a microbenchmark generator for each primitive capability. It can generate tasks with different parameter configurations and across multiple difficulty levels. The AOT compiler takes skills and profiling results as input and packages all compiled artifacts for subsequent execution. These artifacts may include, for example, environment-binding scripts and DAGs for parallel analysis. The compiler is organized as a sequence of modular passes, each of which can be invoked independently.

The JIT optimizer performs runtime optimization using information collected during agent execution, including the conversation history and tool invocations. It supports two operating modes: *autotune* and *real-trace*. In autotune mode, the optimizer drives a headless agent that iteratively edits skill files in an isolated workspace. In real-trace mode, it directly consumes an existing execution history and does not initiate additional agent runs. The optimizer snapshots the state of every optimization round, allowing any round to be inspected, replayed, or rolled back.

SkillVM connects to heterogeneous agents through an adapter layer. It currently supports OpenClaw, Hermes Agent, OpenCode, Pi Coding Agent, Codex, and Claude Code, and can be readily extended to additional agent frameworks. Code solidification requires hooks at two execution points: before and after each LLM invocation. These hooks can be integrated into existing agent frameworks without intrusive modifications. Our current implementation is built on BareAgent.

7 Evaluation

7.1 Setup

Benchmark. We select and extend existing skill benchmarks, including SkillsBench [41] and PinchBench [55], covering diverse agent tasks spanning code generation, data analysis, document creation, and system administration. Each task is paired with at least one skill sourced from mainstream skill sources: Anthropic-skills [7], OpenClaw skills [51], and `skills.sh` [67]. We employ an automated evaluation suite comprising static analysis and LLM-based assessment against predefined criteria [37, 83].

Models. We evaluate SkillVM across eight models spanning three capability tiers to comprehensively assess its effectiveness across varying model capacities. *SOTA*: claude-opus-4.6 [11], deepseek-v3.2 [43]. *Mid-tier*: gemini-3-flash [23], qwen3.5-397b [60], qwen3.5-122b [59], claude-3.5-haiku [5]. *Small*: qwen3-30b [58], devstral-small [48]. On Codex we evaluate two additional models: gpt-5.6-sol and gpt-5.4-mini.

Baselines. We compare SkillVM against three baselines. *No Skill*: the agent executes without any skill loaded. *Original*: the original skill. *Skill-Creator*: the skill optimized by Anthropic's Skill-Creator [10] using claude-opus-4.6.

Harnesses. We select four agent harnesses to evaluate SkillVM performance, each harness offers distinct functionality: *BareAgent*: a minimal harness that injects the compiled skill directly into the system prompt without additional abstraction layers. *OpenCode* [52]: a full-featured code agent harness

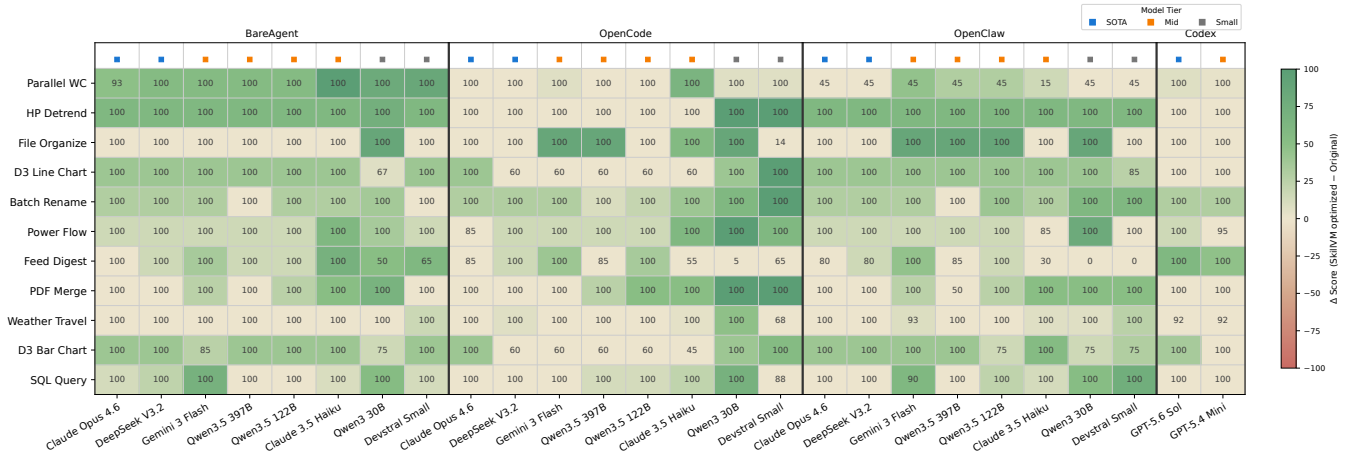


Figure 9. Skill compilation effectiveness. Cell values represent task completion rates after SkillVM optimization, while cell color indicates the score improvement (green) or regression (red) relative to original baseline skills.

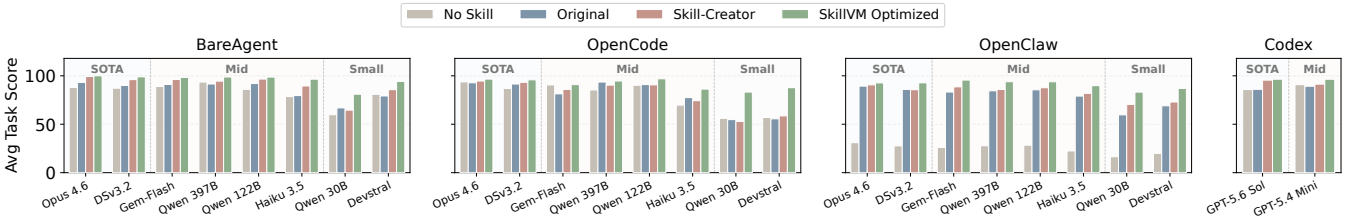


Figure 10. Average task score by skill variant. Four variants are compared across ten models and four harnesses: No Skill, Original, Skill-Creator, and SkillVM-Optimized. Tier boundaries (SOTA / Mid / Small) are shaded.

with batch tool dispatch capabilities and a rich tool set for code execution. *OpenClaw* [51]: a general-purpose agent harness with integrated functional modules designed to support complex multi-step task execution. *Codex* [50]: a SOTA agent harness by OpenAI.

Methodology. For each task, we generate five input instances by varying only task-specific content, such as document text or data values, while keeping the required workflow fixed. We execute each instance three times to account for stochastic LLM inference. We report task score, token consumption, and end-to-end execution latency, averaging each metric first across the three runs and then across the five instances. All experiments are conducted on a Mac Mini M4 with 16GB memory. Unless stated otherwise, all runs share this fully provisioned host environment, so a target reduces to a model–harness pair. §7.6 varies the environment dimension explicitly.

7.2 Skill Compilation Effectiveness

We first evaluate SkillVM's effectiveness in improving task completion rates. We compare SkillVM-optimized skills against unoptimized baseline skills across different tasks, models, and agent harnesses. For SkillVM, the primary gains stem from capability-based compilation and JIT optimization. Figure 9 presents compilation effectiveness across ten models and four harnesses. Cell values represent task completion rates after SkillVM optimization, while cell color indicates the score improvement (green) or regression (red) relative to unoptimized baseline skills.

Figure 9 demonstrates SkillVM's optimization effectiveness (compared with original skills) across diverse models and agent harnesses. Weaker models benefit more substantially from SkillVM optimization. This finding indicates that for most agent tasks, weaker models possess sufficient capability for the logical components but lack proficiency in non-logical aspects, such as generating complex JSON structures or managing environment dependencies. SkillVM's compilation optimization addresses these gaps through primitive capability refinement, thereby improving overall task completion rates. For stronger model–harness configurations, improvements are more modest, as these models already handle tasks competently; SkillVM optimization primarily reduces token consumption and execution latency.

Figure 10 aggregates average task scores across four skill variants per model and harness. SkillVM-optimized skills achieve the highest scores on every model–harness pair. The improvement over the Skill-Creator baseline grows with decreasing model capability: on BareAgent, SkillVM-optimized skills improve over Skill-Creator by 25% for qwen3-30b and 10% for devstral-small. Relative to the no-skill baseline, only 4.5% of tasks have lower completion rates with SkillVM-compiled skills, compared with 15% with the original skills.

Using the original skill, OpenCode and OpenClaw differ by up to 13 points, depending on the model, which shows

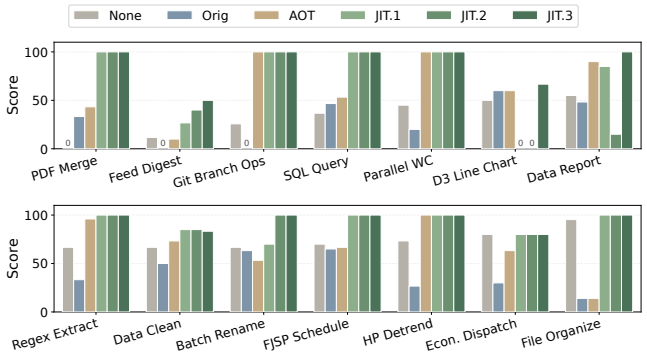


Figure 11. Staged optimization across 14 skill categories. Each group shows six stages: no skill, original skill, AOT-compiled skill, and three rounds of JIT optimization.

that harness behavior alone materially changes outcomes. SkillVM optimization raises scores on both harnesses and reduces this cross-harness gap to at most 5 points.

7.3 Staged Optimization and Cost Breakdown

Figure 11 breaks down the contribution of each optimization stage across 14 skill categories on Qwen3-30B with BareAgent. Each group shows scores at six stages: no skill, original skill, AOT-compiled skill, and up to three rounds of JIT optimization. Tasks used in each round are of the same type but differ in contents.

In 11 of 14 categories, the original skill performs worse than using no skill. After SkillVM's AOT compilation, the average task score improves by 88%. JIT optimization identifies additional capability defects that were not exposed during AOT compilation, further improving performance. After one JIT round, 8 of 14 skills achieve full scores on the evaluation tasks, and after three rounds, 10 of 14 do so.

JIT optimization can introduce regressions. For example, in the Line Chart task, the JIT compiler added an example file to the skill, but the file was too long and triggered parsing errors, which reduced the score. After three rounds of revision guided by prior failure logs, the JIT compiler eventually fixed this issue.

End-to-end cost breakdown. We further analyze the cost incurred across the optimization stages, using claude-sonnet-4.6 as the compiler model. Profiling consumes 1.15M tokens and costs $0.072, with the cost arising from queries to the target model. AOT compilation consumes an average of 90K tokens per skill, costing $0.56. Across the three JIT optimization rounds, the average per-skill costs are 141K tokens ($0.98), 172K tokens ($1.00), and 159K tokens ($0.94), respectively. The costs of AOT compilation and JIT optimization are dominated by calls to the compiler model. Profiling and AOT compilation are one-time costs for a given model–harness pair and skill. Although JIT optimization may be invoked repeatedly as runtime information accumulates, its invocation frequency decreases as the skill converges after successive optimization rounds and enters stable use.

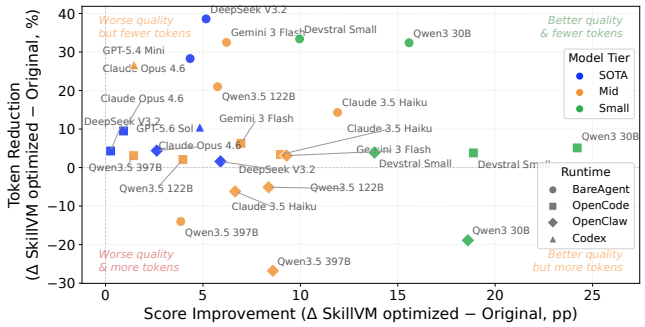


Figure 12. Score gain vs. token savings for SkillVM optimization over the Original baseline. Each point is a model–harness pair. Color = model tier, shape = harness. Most points land in the upper-right quadrant: better quality *and* fewer tokens.

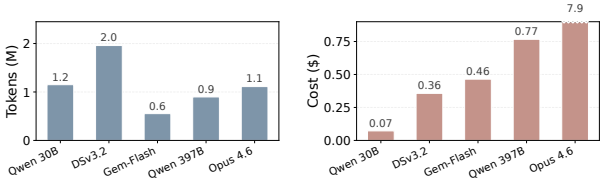


Figure 13. Profiling cost across five models spanning three capability tiers. Left: token consumption. Right: dollar cost (Opus 4.6 truncated).

7.4 Token and Cost Efficiency with Compilation

Figure 12 shows how task success rates and token costs vary across different model tiers and harnesses after SkillVM compilation. For most model–harness combinations, SkillVM simultaneously improves task success and reduces token consumption. For DS-v3.2 and BareAgent pair, we observe token savings approaching 40%. The two Codex configurations also fall in this quadrant, at 4.84 and 1.46 points of score gain with 10.5% and 26.7% fewer tokens. This improvement arises because agent loops incur interaction overhead when correcting environment-related and tool-invocation errors; multiple retries are often required. SkillVM enables models to avoid these error paths entirely, eliminating redundant interactions and saving substantial tokens.

For lifecycle token consumption of a skill, the optimization overhead can be amortized over repeated use, as it is incurred only during compilation, whereas token savings accrue on each subsequent execution.

For weaker models, SkillVM may occasionally consume additional tokens. This occurs because SkillVM improves task completion rates, leading to longer agent execution traces compared to the baseline skill, resulting in modest token increases in edge cases.

7.5 Overhead of Target Profiling

The end-to-end cost breakdown in Section 7.3 shows that target profiling incurs modest overhead. Figure 13 further examines how this overhead varies across model tiers. Because each target runs the same profiling suite, token consumption

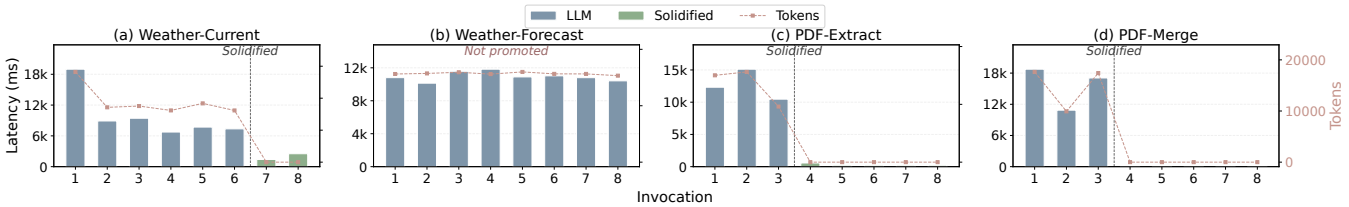


Figure 14. Code solidification across four cases. Blue bars show LLM inference latency, green bars show solidified execution. The dashed line marks the promotion point. Weather-forecast never promotes because the runtime model's code patterns diverge from the AOT prediction, validating the promotion gate as a safety mechanism.

does not systematically increase with model tier, ranging from 0.55 M to 1.96 M tokens. Monetary cost primarily reflects differences in per-token list prices. Even at the upper end, a complete profile costs only $7.92 and is incurred once per model–harness pair.

7.6 Evaluating Environment Binding

The preceding two experiments conducted testing in fully provisioned environments. However, in real-world skill scenarios, missing environment dependencies during agent execution are commonplace. Resolving such environmental issues presents significant challenges for weaker LLMs, frequently resulting in skill execution failures. SkillVM addresses this through an environment binding mechanism in its AOT compilation phase. This mechanism shifts environment dependency installation from the LLM execution phase to pre-execution preprocessing, allowing the LLM to focus solely on skill logic.

Figure 15 compares the performance of different models on the `pdf-create` and `document-xlsx` skills under three environment configurations: complete, with environment binding, and with missing dependencies. Stronger models such as claude-opus-4.6 can autonomously recover from missing dependencies, but at the cost of increased token consumption. For weaker models such as qwen3.5-122b, missing dependencies lead to execution failures. Environment binding restores task scores to the fully provisioned baseline and substantially reduces token consumption.

7.7 Evaluating Concurrency Extraction

Figure 16 shows that SkillVM achieves up to 3.16× speedup over the sequential variant of the same skill across nine tasks. Script-level parallelism achieves a 1.23× mean speedup, because the data processing it overlaps accounts for less than 30% of end-to-end latency, and the remaining agent-loop inference stays serial. Tool-level parallelism attains the highest mean speedup because collapsing independent tool calls into a single batched turn removes model round-trips as well as the tool call latency itself. Agent-level parallelism reaches 2.16× speedup on average, below tool-level despite its coarser grain. Sub-agent spawn cost and the join at the end of the workflow bound the gain, and the critical path is set by the slowest sub-agent rather than the average one. It nonetheless cuts token consumption by 60% on average,

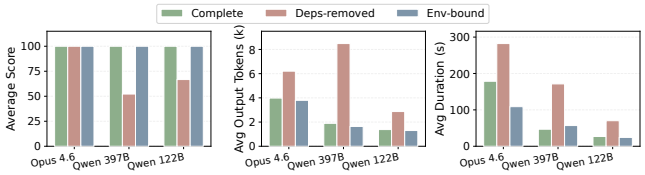


Figure 15. Environment binding restores task completion rate, token efficiency, and execution speed. From left to right: average score, average token use, and average duration.

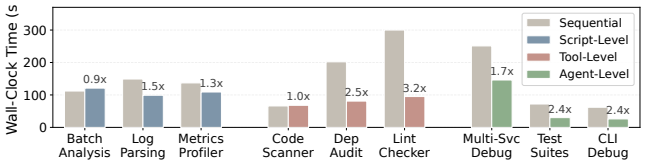


Figure 16. Performance of different parallelization strategies across nine tasks. Bars show sequential, script-level, tool-level, and agent-level execution.

since each sub-agent carries only the context its subtask requires. Task scores are unchanged across all three variants, so the rewrites preserve correctness.

7.8 Evaluating Code Solidification

We further analyze whether adopting code solidification mechanisms from JIT optimization can accurately identify reusable code segments and improve agent execution efficiency. As shown in Figure 14, we conduct a detailed analysis on two canonical skills (weather, document-pdf). For the two PDF tasks, despite variations in task inputs, the LLM-generated code matches the code signature. Therefore, SkillVM can apply JIT compilation optimization, directly generating solidified code using the template and input parameters. For PDF-extract tasks, JIT optimization reduces execution time from 10,469–15,116 ms to 206–568 ms, achieving a 19–50× speedup. For the two weather tasks, weather-current requires external weather API calls, the speedup is bounded by network latency, reducing execution time from 9,000 ms to 2,000 ms, yielding a 5× improvement. In contrast, weather-forecast uses more flexible formats, so the generated code structures diverge from the code signature and fail to match. Consequently, all eight invocations rely on LLM generation to avoid invoking incorrect code. If solidified code fails to execute, SkillVM demotes the candidate and re-enables LLM code generation.

8 Related Work

LLM Prompt and Workflow Optimization. Several prior studies have focused on optimizing LLM prompts and workflows [3, 18, 26, 30, 33, 53, 56, 61, 65, 74, 79, 80, 82, 84, 85]. DSPy [38] compiles declarative LM programs by learning module parameters from examples against a user-defined task metric. Cognify [32] hierarchically searches over prompt, operator, and workflow-structure transformations to balance output quality, monetary cost, and latency. At user request level, LLMCompiler [39] constructs a dependency graph of tool calls and schedules independent calls in parallel.

SkillVM differs from these systems in two ways. First, SkillVM optimizes per skill rather than per task. DSPy-style optimizers specialize an LM program for one application, driven by a training set and an evaluation metric defined for that task, and the tuned program serves that task alone. SkillVM compiles a skill against the measured capability profile of its target, with no per-task training data, so every task that invokes the skill benefits from one compilation. Second, SkillVM restructures agent execution rather than tuning the prompt and skill content. SkillVM extracts parallelism across multiple levels and solidifies repeated LLM work into deterministic code that bypasses inference, transformations that a prompt optimizer cannot express.

Self-Evolving Skill Optimization. Recent works also optimize skills through execution feedback [15, 17, 25, 28, 34, 42, 46, 47, 54, 66, 81]. SkillOpt [76] converts scored rollouts into bounded, validated edits to a skill document. SkillAdaptor [78] attributes failures to individual trajectory steps before applying targeted updates. Like SkillVM's adaptive recompilation, these systems iteratively propose, evaluate, and retain skill modifications.

SkillVM differs from these systems along two dimensions.

Efficiency. These systems primarily revise skill content to improve task performance, whereas SkillVM also focuses on execution cost and latency. Its runtime solidifies recurring model-generated code paths into deterministic code, and its compiler explores potential parallel opportunities.

Target awareness. Existing skill-optimization loops learn from trajectories produced by a particular model and harness and do not explicitly model differences across target executors. SkillVM instead profiles each target's model, harness, and environment and compiles one source skill into a target-specific variant.

9 Discussion

Correctness and non-determinism of LLM. Integrating LLMs into the compiler and optimizer of SkillVM enables the system to process natural language inputs, but also introduces non-determinism and potential execution errors. We mitigate these LLM-induced risks across three complementary levels. First, we validate properties that are mechanically checkable: workflow DAGs must contain no dangling nodes, and environment-binding scripts as well as solidification templates are pre-executed to verify syntax and eliminate runtime errors. Second, SkillVM leverages the LLM's ability to retry and recover from imperfect compiled artifacts. When the current skill variant performs poorly, the observed failure and recovery traces trigger another round of JIT optimization. Similarly, if the code generated at runtime repeatedly fails to match a solidification candidate, SkillVM regenerates the candidate based on the observed code pattern. Thus, an LLM-generated artifact is not treated as a fixed compilation result, but can be iteratively revised using runtime feedback. Third, we evaluate tasks using deterministic tests and static analysis if a property is machine-checkable, reserving LLM judges for semantic or open-ended criteria. We also use repeated runs to account for stochastic variation.

Generality beyond skills. Our evaluation is grounded in today's skill ecosystem, but SkillVM addresses a broader problem: reusable agent procedures are deployed across targets whose models, harnesses, and environments their authors cannot anticipate. Skills may be replaced by other abstractions, yet agents will still require task-specific guidance and a harness that mediates tools and the host environment. Model advances may narrow capability gaps, but they do not standardize harnesses or deployment environments. As long as targets remain heterogeneous, SkillVM's target-aware specialization applies beyond the current skill abstraction.

10 Conclusion

Skills have emerged as a new code form in the agent era. However, after analyzing over 100,000 skills, we discover a significant mismatch between skills and underlying LLMs. To address this, we propose SkillVM, a compilation and runtime system specifically tailored for skills. SkillVM leverages compilation techniques such as AOT and JIT to optimize skill structure, extract environment dependencies, and identify parallelism opportunities. As models and agent harnesses continue to diversify, skill compilation provides a systematic pathway to transform skills into portable components, rather than fragile prompts. The source code of SkillVM is publicly available at https://github.com/SJTU-IPADS/SkVM.

Acknowledgments

We sincerely thank our shepherd Tim Harris and the anonymous reviewers of SOSP 2026, whose reviews, feedback, and suggestions have significantly strengthened our work. We also thank Amanda XIE for helping us access AI resources. This work is supported in part by the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (JYB2025XDXM113), China National Natural Science Foundation (No. 62472279, 62602004). We thank Huawei Advanced Research Center for Agent-Oriented OS for their support. Yubin Xia is the corresponding author.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[2] Agent Skills Initiative. 2025. Agent Skills Specification. https://agentskills.io/specification. Open SKILL.md format for agent skill portability; accessed: 2026-02-06.

[3] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. In *International Conference on Learning Representations (ICLR)*.

[4] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. 2006. *Compilers: Principles, Techniques, and Tools* (2nd ed.). Addison-Wesley.

[5] Anthropic. 2024. Claude 3.5 Haiku. https://www.anthropic.com/news/3-5-models-and-computer-use. Accessed: 2026-04-02.

[6] Anthropic. 2025. Agent Skills. https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview. Accessed: 2026-02-02.

[7] Anthropic. 2025. Anthropic Skills Repository. https://github.com/anthropics/skills. Accessed: 2026-03-22.

[8] Anthropic. 2025. Equipping Agents for the Real World with Agent Skills. https://claude.com/blog/equipping-agents-for-the-real-world-with-agent-skills. Accessed: 2026-02-06.

[9] Anthropic. 2025. Extend Claude with Skills. https://code.claude.com/docs/en/skills. Claude Code skill mechanism; accessed: 2026-02-06.

[10] Anthropic. 2025. Skill Creator. https://github.com/anthropics/skills/tree/main/skills/skill-creator. Accessed: 2026-03-23.

[11] Anthropic. 2026. Claude 4.6 Opus. https://www.anthropic.com/claude/opus. Accessed: 2026-04-02.

[12] Anthropic. 2026. Claude Code. https://github.com/anthropics/claude-code. Accessed: 2026-04-01.

[13] Matthew Arnold, Stephen J. Fink, David Grove, Michael Hind, and Peter F. Sweeney. 2005. A Survey of Adaptive Optimization in Virtual Machines. *Proc. IEEE* 93, 2 (2005), 449–465. doi:10.1109/JPROC.2004.840305

[14] John Aycock. 2003. A brief history of just-in-time. *ACM computing surveys (CSUR)* 35, 2 (2003), 97–113.

[15] Xiaoyi Bao, Yuanzhen Xie, Yunzhi Tan, Jinghang Gu, Zhongqing Wang, Chu-Ren Huang, Bo Hu, and Zang Li. 2026. SkillMentor: LLM Agent Self-Evolution via Learning Blind-Spot Diagnosis. *arXiv preprint arXiv:2607.27360* (2026).

[16] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems (NeurIPS)* 33 (2020), 1877–1901.

[17] Kunfeng Chen, Qihuang Zhong, Juhua Liu, and Bo Du. 2026. SkillCAT: Contrastive Assessment and Topology-Aware Skill Self-Evolution for LLM Agents. *arXiv preprint arXiv:2606.13317* (2026).

[18] Ching-An Cheng, Allen Nie, and Adith Swaminathan. 2024. Trace is the Next AutoDiff: Generative Optimization with Rich Feedback, Execution Traces, and LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[19] ClawHub. 2025. ClawHub: Agent Skills Marketplace. https://clawhub.ai. Accessed: 2026-03-22.

[20] Coze. 2025. Plugin Development Guide. https://www.coze.com/docs/guides/plugin?_lang=en. Coze bot plugins and skills; accessed: 2026-02-06.

[21] Timothy Cramer, Richard Friedman, Terrence Miller, David Seberger, Robert Wilson, and Mario Wolczko. 1997. Compiling Java just in time. *Ieee micro* 17, 3 (1997), 36–43.

[22] Cursor. 2025. Agent Skills. https://cursor.com/docs/context/skills. Cursor IDE skill mechanism; accessed: 2026-02-06.

[23] DeepMind. 2026. Our latest Gemini 3 model that helps you bring any idea to life - faster. https://deepmind.google/models/gemini/flash/. Accessed: 2026-04-02.

[24] L. Peter Deutsch and Allan M. Schiffman. 1984. Efficient Implementation of the Smalltalk-80 System. In *Proceedings of the 11th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*. 297–302. doi:10.1145/800017.800542

[25] Kexin Ding, Yang Zhou, Can Jin, Feng Tong, Mu Zhou, and Dimitris N. Metaxas. 2026. Agent Skill Evaluation and Evolution: Frameworks and Benchmarks. *arXiv preprint arXiv:2606.11435* (2026).

[26] Chrisantha Fernando, Dylan Sunil Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. 2024. Promptbreeder: Self-Referential Self-Improvement via Prompt Evolution. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, Vol. 235. 13481–13544.

[27] FreedomIntelligence. 2025. OpenClaw Medical Skills. https://github.com/FreedomIntelligence/OpenClaw-Medical-Skills. Accessed: 2026-03-23.

[28] Srishti Gautam, Arjun Radhakrishna, and Sumit Gulwani. 2026. SkillAxe: Sharpening LLM-Authored Agent Skills Through Evaluation-Guided Self-Refinement. *arXiv preprint arXiv:2606.10546* (2026).

[29] Google. 2025. Agent Skills. https://antigravity.google/docs/skills. Google Antigravity coding agent skills; accessed: 2026-02-06.

[30] Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. 2024. Connecting Large Language Models with Evolutionary Algorithms Yields Powerful Prompt Optimizers. In *International Conference on Learning Representations (ICLR)*.

[31] Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. 2026. SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering? *arXiv preprint arXiv:2603.15401* (2026).

[32] Zijian He, Reyna Abhyankar, Vikranth Srivatsa, and Yiying Zhang. 2025. Cognify: Supercharging Gen-AI Workflows With Hierarchical Autotuning. *arXiv preprint arXiv:2502.08056* (2025). doi:10.48550/arXiv.2502.08056

[33] Shengran Hu, Cong Lu, and Jeff Clune. 2025. Automated Design of Agentic Systems. In *International Conference on Learning Representations (ICLR)*.

[34] Siyuan Huang, Pengyu Cheng, Haotian Liu, Tao Chen, Yihao Liu, Jingwei Ni, Shijie Zhou, Ziyi Yang, et al. 2026. Skill Self-Play: Pushing the Frontier of LLM Capability with Co-Evolving Skills. *arXiv preprint arXiv:2607.22529* (2026).

[35] Kazuaki Ishizaki, Motohiro Kawahito, Toshiaki Yasue, Hideaki Komatsu, and Toshio Nakatani. 2000. A study of devirtualization techniques for a Java Just-In-Time compiler. In *Proceedings of the 15th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*. 294–310.

[36] Jan Martin Jansen, Pieter Koopman, and Rinus Plasmeijer. 2007. From interpretation to compilation. In *Central European Functional Programming School*. Springer, 286–301.

[37] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations (ICLR)*.

[38] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, et al. 2023. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. *arXiv preprint arXiv:2310.03714* (2023).

[39] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. 2024. An LLM Compiler for Parallel Function Calling. In *International Conference on Machine*

Learning (ICML).

[40] LangChain. 2024. LangGraph: Build Resilient Language Agents as Graphs. https://github.com/langchain-ai/langgraph. Accessed: 2026-03-22.

[41] Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, et al. 2026. SkillsBench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670* (2026).

[42] Huawei Lin, Peng Li, Jie Song, Fuxin Jiang, and Tieying Zhang. 2026. MUSE-Autoskill: Self-Evolving Agents via Skill Creation, Memory, Management, and Evaluation. *arXiv preprint arXiv:2605.27366* (2026).

[43] Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, et al. 2025. DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models. *arXiv preprint arXiv:2512.02556* (2025).

[44] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2023. Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing. *Comput. Surveys* 55, 9 (2023), 1–35. arXiv:2107.13586, 2021.

[45] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. 2024. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations (ICLR).*

[46] Yuxuan Liu, Zhaochen Su, Lingyun Xie, Yuhao Zhang, Qing Zong, Jiahe Guo, Zhongwei Xie, Yiyan Ji, et al. 2026. SkillRevise: Improving LLM-Authored Agent Skills via Trace-Conditioned Skill Revision. *arXiv preprint arXiv:2606.01139* (2026).

[47] Yuxuan Liu, Zhaochen Su, Yuhao Zhang, Jiahe Guo, Zhongwei Xie, Huihao Jing, Lingyun Xie, Qing Zong, et al. 2026. Rethinking Self-Evolving Agent Skills: Feedback Dynamics over Multiple Rounds. *arXiv preprint arXiv:2608.02636* (2026).

[48] Mistral AI. 2026. Devstral Small 2507. https://huggingface.co/mistralai/Devstral-Small-2507. Accessed: 2026-04-02.

[49] OpenAI. 2023. Function Calling and Other API Updates. https://openai.com/index/function-calling-and-other-api-updates/. Accessed: 2026-02-06.

[50] OpenAI. 2025. OpenAI Codex. https://developers.openai.com/codex. Accessed: 2026-02-06.

[51] OpenClaw. 2025. OpenClaw: Personal AI Assistant. https://github.com/openclaw/openclaw. Accessed: 2026-03-22.

[52] OpenCode. 2025. OpenCode: Open-Source AI Coding Agent. https://github.com/opencode-ai/opencode. Accessed: 2026-03-22.

[53] Krista Opsahl-Ong, Michael J Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. 2024. Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP).* 9340–9366.

[54] Siran Peng, Cuiyu Yang, Tianyu Fu, Tianshuo Zhang, Haoyuan Zhang, Weisong Zhao, Anyang Su, Minghui Wu, et al. 2026. Self-Supervised Skill Optimization. *arXiv preprint arXiv:2607.28777* (2026).

[55] PinchBench. 2026. PinchBench: Benchmarking LLM Models as Coding Agents. https://github.com/pinchbench/skill. Accessed: 2026-02-06.

[56] Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. 2023. Automatic Prompt Optimization with "Gradient Descent" and Beam Search. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP).* 7957–7968.

[57] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs. In *International Conference on Learning Representations (ICLR).*

[58] Qwen. 2026. Qwen-3-30B. https://huggingface.co/Qwen/Qwen3-30B-A3B. Accessed: 2026-04-02.

[59] Qwen. 2026. Qwen-3.5-122B. https://huggingface.co/Qwen/Qwen3.5-122B-A10B. Accessed: 2026-04-02.

[60] Qwen. 2026. Qwen-3.5-397B. https://huggingface.co/Qwen/Qwen3.5-397B-A17B. Accessed: 2026-04-02.

[61] Jon Saad-Falcon, Adrian Gamarra Lafuente, Shlok Natarajan, Nahum Maru, Hristo Todorov, Etash Guha, E. Kelly Buchanan, Mayee Chen, Neel Guha, Christopher Ré, and Azalia Mirhoseini. 2025. Archon: An Architecture Search Framework for Inference-Time Techniques. In *Proceedings of the 42nd International Conference on Machine Learning (ICML).*

[62] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems* 36 (2023), 68539–68551.

[63] Manuel Serrano. 2018. JavaScript AOT compilation. *ACM SIGPLAN Notices* 53, 8 (2018), 50–63.

[64] Manuel Serrano. 2021. Of JavaScript AOT compilation performance. *Proceedings of the ACM on Programming Languages* 5, ICFP (2021), 1–30.

[65] Yu Shang, Yu Li, Keyu Zhao, Likai Ma, Jiahe Liu, Fengli Xu, and Yong Li. 2025. AgentSquare: Automatic LLM Agent Search in Modular Design Space. In *International Conference on Learning Representations (ICLR).*

[66] Yifei Shen, Bo Li, and Xinjie Zhang. 2026. SkillOpt-Lite: Better and Faster Agent Self-Evolution via One Line of Vibe. *arXiv preprint arXiv:2607.03451* (2026).

[67] Skills.sh. 2025. Skills.sh: Agent Skills Registry. https://skills.sh. Accessed: 2026-03-22.

[68] Bjarne Stroustrup. 2013. *The C++ programming language.* Pearson Education.

[69] Toshio Suganuma, Takeshi Ogasawara, Mikio Takeuchi, Toshiaki Yasue, Motohiro Kawahito, Kazuaki Ishizaki, Hideaki Komatsu, and Toshio Nakatani. 2000. Overview of the IBM Java just-in-time compiler. *IBM systems Journal* 39, 1 (2000), 175–193.

[70] Bill Venners. 1998. *Inside the Java Virtual Machine.* McGraw-Hill.

[71] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. 2024. A Survey on Large Language Model based Autonomous Agents. *Frontiers of Computer Science* 18, 6 (2024), 186345. doi:10.1007/s11704-024-40231-1

[72] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, et al. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In *Conference on Language Modeling (COLM).*

[73] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).

[74] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large Language Models as Optimizers. In *International Conference on Learning Representations (ICLR).*

[75] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems* 37 (2024), 50528–50652.

[76] Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, et al. 2026. SkillOpt: Executive Strategy for Self-Evolving Agent Skills. *arXiv preprint arXiv:2605.23904* (2026).

[77] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations (ICLR)* (2023).

[78] Zhuoyun Yu, Xin Xie, Wuguannan Yao, Chenxi Wang, Lei Liang, Xiang Qi, and Shumin Deng. 2026. SkillAdaptor: Self-Adapting Skills for LLM Agents from Trajectories. *arXiv preprint arXiv:2606.01311* (2026).

[79] Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. 2025. Optimizing Generative AI by Backpropagating Language Model Feedback. *Nature* 639, 8055 (2025), 609–616. doi:10.1038/s41586-025-08661-4

[80] Guibin Zhang, Luyang Niu, Junfeng Fang, Kun Wang, Lei Bai, and Xiang Wang. 2025. Multi-Agent Architecture Search via Agentic Supernet. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*.

[81] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, et al. 2026. CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification. *arXiv preprint arXiv:2604.01687* (2026).

[82] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. 2025. AFlow: Automating Agentic Workflow Generation. In *International Conference on Learning Representations (ICLR)*.

[83] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36.

[84] Han Zhou, Xingchen Wan, Ruoxi Sun, Hamid Palangi, Shariq Iqbal, Ivan Vulić, Anna Korhonen, and Sercan Ö. Arık. 2026. Multi-Agent Design: Optimizing Agents with Better Prompts and Topologies. In *International Conference on Learning Representations (ICLR)*.

[85] Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. 2024. Language Agents as Optimizable Graphs. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, Vol. 235. 62743–62767.

Appendix A Adaptive Recompilation

This appendix is supplementary material that has not been peer-reviewed.

This appendix specifies the adaptive recompilation algorithm: the optimization loop (§A.1) and the diagnosis contract it enforces (§A.2).

A.1 Optimization Loop

Algorithm 1 operates on evidence collected in rounds. Each round executes every task R times and groups repeats by task, preserving whether a criterion recurs within one task or across multiple tasks. FailureProfile summarizes that evidence without model inference, recording per task and criterion how many clean runs failed it. A run is clean unless its harness status reports a timeout, an adapter crash, or a parse failure, in which case it is excluded from scoring rather than scored 0, and a task with no clean run is Tainted. Round k edits the skill produced by round $k-1$. All evaluated rounds remain eligible until selection at the end of the loop. The implementation defaults are $R = 2$, $K = 3$, and $\tau = 0.95$.

Round selection. Select applies the admission criteria to candidates evaluated on a runnable task set. Let p_k denote $\textsc{Primary}(E_k)$, the unweighted mean of the per-task means over clean runs, measured on the held-out test set when a separate test set is available and on the training set otherwise. Let c_k denote the target-agent cost of the task set under that round's skill. A round is admitted if it improves on the baseline by at least $\delta = 0.1$, or if its score is within δ of the baseline and its cost is at least $\gamma = 15\%$ lower. The value of δ is above the measured per-task score deviation of 0.03 to 0.08 on stable skills. A candidate is excluded if any baseline task regresses by more than $\rho = 0.09$ or has no clean run. Surviving rounds are ordered by score outside the δ band, by cost within the band, and by round number to resolve remaining ties. If the baseline meets τ or no candidate survives, Select returns the baseline. Each exclusion is recorded with the criterion that caused it.

Algorithm 1 Adaptive recompilation

Require: skill S_0, task set T, target $\langle$model m, harness $h\rangle$
Require: repeats $R = 2$, max rounds $K = 3$, convergence $\tau = 0.95$
Ensure: the shipped skill

$E_0 \leftarrow \textsc{Execute}(S_0, T, R)$ ▷ R independent runs of each task
if $\textsc{Primary}(E_0) \geq \tau$ **then**
  return S_0 ▷ baseline meets convergence threshold
end if
$rounds \leftarrow [\langle 0, S_0, E_0\rangle]$
for $k \leftarrow 1$ **to** K **do**
  $F \leftarrow \textsc{FailureProfile}(E_{k-1})$ ▷ deterministic, no inference
  if every task in F is Tainted **then**
    abort InfraBlocked
  end if
  $D \leftarrow \textsc{Diagnose}(S_{k-1}, E_{k-1}, F, Hist, \Pi(m, h))$
  if D abstains **then**
    record $\langle k, D.reason\rangle$, **break**
  end if
  $S_k \leftarrow D.editedSkill$ ▷ the diff defines the proposal
  $E_k \leftarrow \textsc{Execute}(S_k, T, R)$
  $rounds \leftarrow rounds +\!\!+ [\langle k, S_k, E_k\rangle]$
  if $\textsc{Primary}(E_k) \geq \tau$ **then break**
  end if
end for
$k^* \leftarrow \textsc{Select}(rounds)$ ▷ admission gate, below
return S_{k^*}

Evidence sources. Admission by re-execution requires a task set with graded criteria. SkillVM also accepts recorded deployment logs, which contain failing inputs and observed failure signals but no fixtures or runnable task definitions. This path performs one diagnosis pass and returns any edit produced by the agent without applying the admission gate.

A.2 Diagnosis Contract

Diagnosis is performed by a headless coding agent. The agent receives a writable copy of the skill directory and a read-only evidence tree containing per-task score summaries, full conversation logs, capped work-directory snapshots, and the root causes and outcomes of previous rounds. It also receives the target's measured capability profile. The agent edits files in place; the resulting diff is the proposal. It also

emits a structured submission containing a root-cause diagnosis, confidence score, and a generality justification for each change.

The contract permits an edit only when all three conditions below hold. Otherwise, the agent abstains and records the failure category.

- **C1, recurrence.** The same evaluation criterion fails in at least two clean runs of one task, or in at least two distinct tasks. Failures of different criteria across two repeats do not satisfy C1.
- **C2, generality.** The proposed fix can be stated without reference to any literal drawn from the failing task, and the agent can name a different plausible task on the same skill that the same change would help. A change whose generality cannot be articulated must be removed from the proposal.
- **C3, no trade-off.** An aggregate improvement must not come at the expense of a task that already succeeds. The agent identifies all tasks that currently pass and assesses whether the proposed edit risks lowering any of their scores. If it identifies such a risk, it must narrow the edit, abstain, or explicitly name the affected task and justify why the edit is unlikely to cause a regression.

C1 and C2 determine whether the failure is classified as systematic. C3 states the editing objective and is judged rather than proven before the edit. The per-task admission gate enforces it afterwards at the looser bound of ρ, so the property an admitted round actually carries is that no task regressed by more than ρ. With the default $R = 2$, C1 is satisfied within a single task only if both runs are clean and fail the same evaluation criterion; increasing R or observing the criterion across tasks provides additional evidence.

Appendix B Code Solidification

This appendix is supplementary material that has not been peer-reviewed.

Algorithm 2 specifies the promotion and demotion behavior of §5.2.2. Promotion requires $N = 3$ consecutive runs in which the model's generated code matches the candidate's code signature; a non-matching run resets the counter. The runtime attempts to serve a promoted candidate only when parameter extraction succeeds and the instantiated template has no remaining holes. Otherwise, it defers to the model. A successful serve replaces the code the model would have generated, not the model itself: the template's output is returned to the model as a tool result, and the model either accepts it as the answer or reports it as wrong.

The fallback counter is cumulative and is not reset after a successful serve. If a demoted candidate later regains promoted status, its next failed serve therefore satisfies the demotion threshold again.

Algorithm 2 Solidification: promotion, bypass, and demotion

Require: promotion threshold $N = 3$, demotion threshold $M = 3$
Observe ▷ after every agent run
for each candidate q with $\neg q.promoted$ **do**
  if some tool call in the run matches $q.codeSignature$ **then**
    $q.consec \leftarrow q.consec + 1$
    if $q.consec \geq N$ **then**
      $q.promoted \leftarrow$ **true**
    end if
  else
    $q.consec \leftarrow 0$ ▷ any non-matching run resets
  end if
end for
Serve ▷ before each model call, on user prompt P
for each promoted q whose keywords occur in P **do**
  $params \leftarrow \textsc{Extract}(P, q)$ ▷ pattern match first, model fallback
  if *params* incomplete or template has holes **then**
    continue ▷ defer to the model
  end if
  $out \leftarrow \textsc{Execute}(\textsc{Instantiate}(q.template, params))$
  if $out.ok \wedge \textsc{Accept}(out, P)$ **then**
    return *out* ▷ code generation skipped
  end if
  $q.fallbacks \leftarrow q.fallbacks + 1$
  if $q.fallbacks \geq M$ **then**
    $q.promoted \leftarrow$ **false**, $q.consec \leftarrow 0$
  end if
end for
return passthrough ▷ normal model path

The runtime checks whether the template executes successfully and detects explicit failures such as runtime exceptions. Semantic correctness is checked by the model. Successful execution shows only that a template ran without error. It does not guarantee that the output is correct. For example, a template that produces an off-by-one error in a computed range would still pass the execution check. To detect such errors, Accept returns the output to the model for validation before it is used. If the model rejects the output, the rejection is treated as a fallback and increments the same counter used for execution failures. Repeated rejections therefore cause the template to be retired.